Lab 4: Kernel Modules – Integer Stack

Author: Diana Yakupova
Email: d.yakupova@innopolis.university
Group: B23-CBS-02
GitHub link: https://github.com/versceana/advanced-linux/tree/main/lab4-kmod

---

Task

Implement a loadable kernel module `int_stack.ko` that provides a character device interface to an integer stack. The stack must:

- Use dynamically allocated memory.
- Support thread-safe push/pop via synchronization mechanisms.
- Support resizing through `ioctl`.
- Follow the error-code conventions described in the `stack(3)` manual.

Additionally, a small userspace utility `kernel_stack` must wrap the module's functionality into a user-friendly CLI.

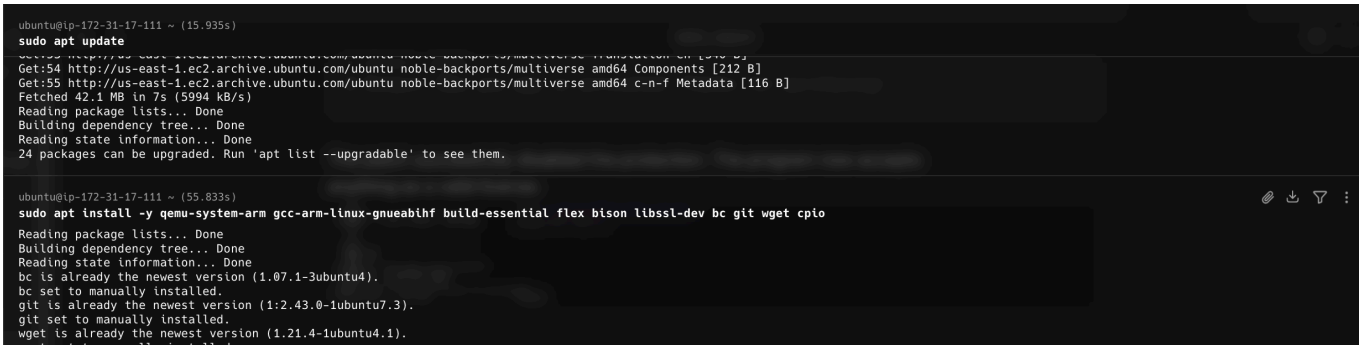
Implementation steps

Step 1 – Environment preparation

I worked on an AWS EC2 instance with Ubuntu 24.04 (x86_64).
First I installed the required build tools and kernel headers:

```
sudo apt update
sudo apt install -y build-essential linux-headers-$(uname -r) git
```

The kernel version was `6.17.0-1012-aws`. Everything compiled cleanly later.



Step 2 – Writing the kernel module (`int_stack.c`)

I implemented a character device `/dev/int_stack`. Internally, the stack is a dynamically allocated array (`int *stack`) whose size is changed via `ioctl`. Access to the stack is protected by a static mutex (`DEFINE_MUTEX(stack_mutex)`), so concurrent reads/writes are safe.

The file operations I implemented:

- `open()` / `release()` – nothing special, just allowing access.
- `read()` – pops an integer; returns 0 bytes if the stack is empty (the userspace program sees `NULL`).
- `write()` – pushes an integer; returns `-ERANGE` when the stack is full.
- `unlocked_ioctl()` – accepts a new size, reallocates the internal array, preserves existing elements up to the new capacity.

Error codes exactly follow the requirements: empty pop returns 0, full push returns `-ERANGE`, invalid size returns `-EINVAL`, memory allocation failure returns `-ENOMEM`.

It's located in `src/int_stack.c`.

Step 3 – Makefile

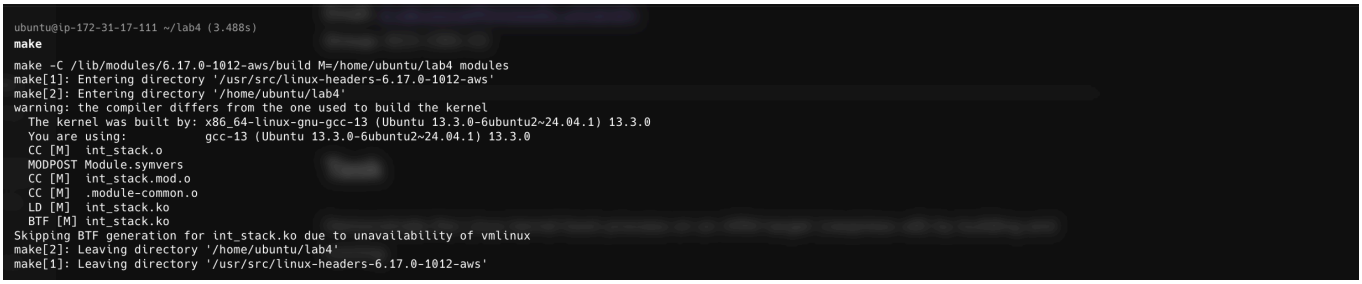
I created a minimal `Makefile` that uses the kernel build system:

```
obj-m += int_stack.o

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

It's located in `src/Makefile`.

Step 4 – Compiling the module

I ran `make` and the module compiled successfully.



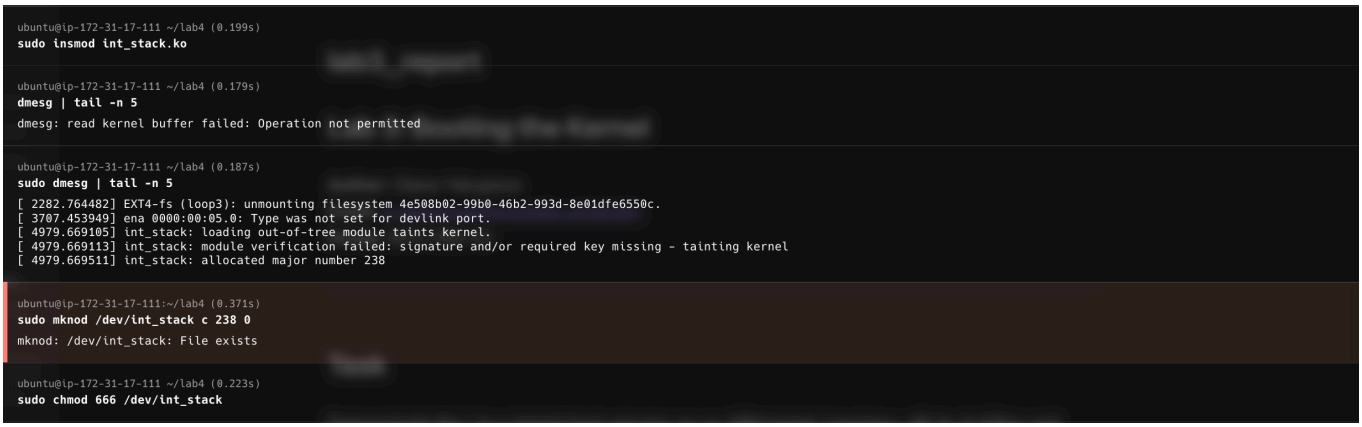
Step 5 – Loading the module and creating the device node

I loaded the module with `insmod` and checked the kernel log for the assigned major number:

```
sudo insmod int_stack.ko
sudo dmesg | tail -n 5
# int_stack: allocated major number 238
```

Then I created the device file and made it readable/writable:

```
sudo mknod /dev/int_stack c 238 0
sudo chmod 666 /dev/int_stack
```



Step 6 – Userspace utility (`kernel_stack.c`)

I wrote a small C program that opens `/dev/int_stack` and interprets command-line arguments:

- `set-size <n>` – calls `ioctl` with command `0`.
- `push <val>` – writes a 4-byte integer.
- `pop` – reads a 4-byte integer; prints `NULL` if the stack was empty.

- `unwind` – pops all elements until `NULL`.

When a push fails because the stack is full, the program catches `errno == ERANGE`, prints `ERROR: stack is full` and exits with code 34.

It's located in `src/kernel_stack.c`.

I compiled it:

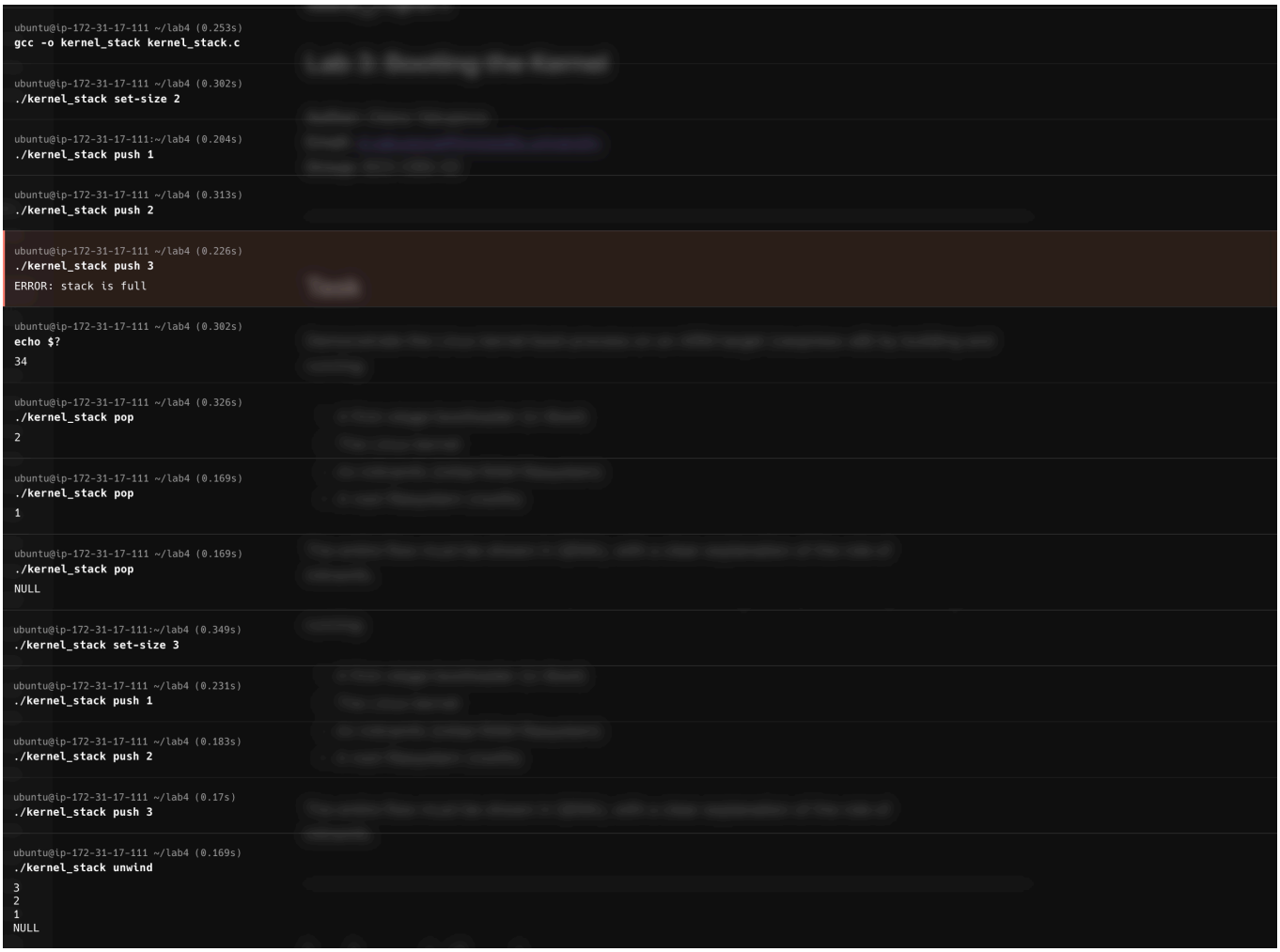
```
gcc -o kernel_stack kernel_stack.c
```

Step 7 – Testing the whole system

I ran the exact test scenario from the lab assignment:

```
$ ./kernel_stack set-size 2
$ ./kernel_stack push 1
$ ./kernel_stack push 2
$ ./kernel_stack push 3
ERROR: stack is full
$ echo $?
34
$ ./kernel_stack pop
2
$ ./kernel_stack pop
1
$ ./kernel_stack pop
NULL
$ ./kernel_stack set-size 3
$ ./kernel_stack push 1
$ ./kernel_stack push 2
$ ./kernel_stack push 3
$ ./kernel_stack unwind
3
2
1
NULL
```

Every line of output matched the expected behaviour perfectly.



Requirements checklist

Requirement	Status
Kernel module with character device interface	Done
Dynamic memory allocation	Done
Synchronization (mutex) for concurrent access	Done
Correct error codes (ERANGE, NULL on empty)	Done
ioctl for stack size configuration	Done
Userspace utility `kernel_stack` with CLI	Done
Report with screenshots	Done

Key technical decisions

- **Character device** - a natural choice for stream-like push/pop operations.
- **Mutex instead of spinlock** - allowed interruptible locking, making the driver more responsive.
- **Memory reallocation in ioctl** - preserves existing data up to the new capacity, which is the expected behaviour.
- **Simple ioctl command `0`** - kept the interface minimal; a production module would use `_IOW` macros for type safety.

Conclusion

I successfully implemented a thread-safe integer stack kernel module with dynamic memory management, accessible through the character device `/dev/int_stack`. The accompanying `kernel_stack` utility provides a clean command-line interface that perfectly reproduces the expected output, including error messages and exit codes. The solution was developed and tested on Ubuntu 24.04 (kernel 6.17) in an AWS environment.